

PEAT

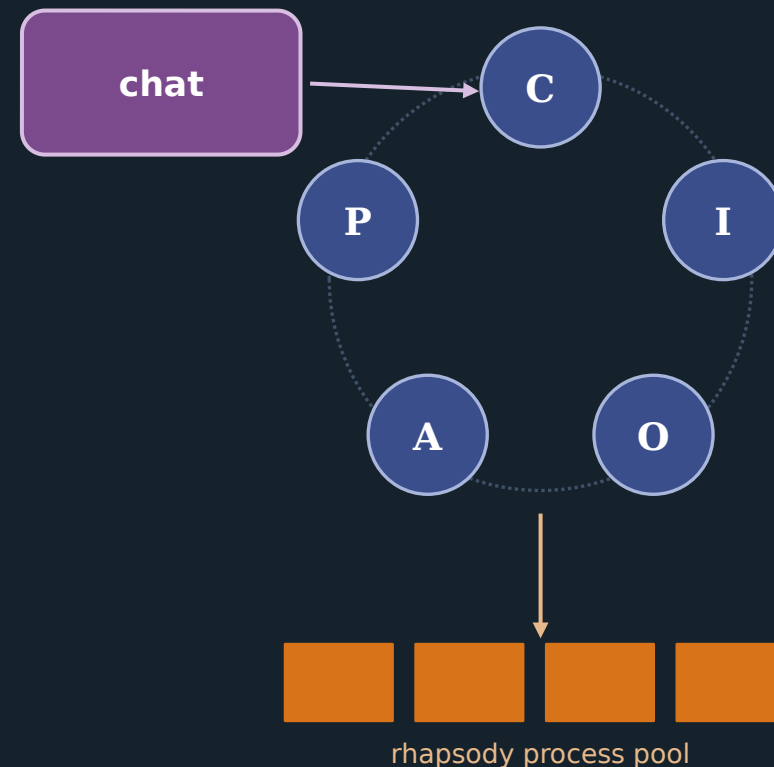
A chat agent that runs real redesign campaigns

A LangGraph loop whose long work leaves the process — and a chat that keeps talking while it does

Worked example, measured: 1OIL (P22088, 320 aa) · 2 rounds · 11 designs · pLDDT 81.66 → 81.79

- FUNCTIONALITY** the new tasks it makes possible
- PERFORMANCE** what it costs to run, and what we never measured
- USABILITY** what it is usable as

Lab code walk · main @ 7e72b74 · ~12.0k lines backend, 6527 lines of tests



One prompt, a whole campaign



What came back, for real (campaign s-tutp87hw)

- 1OIL — P22088, Burkholderia cepacia lipase, 320 residues, resolved from the prompt alone
- lead design G147A at pLDDT 81.79, RMSD 3.75 Å to the crystal structure
- 11 designs scored on 10 metrics; 114 score rows written
- a Markdown summary, a .docx, a sortable ensemble table, and a Mol* view of the lead

And what that is not

- The variants are heuristic single-point proposals, not ProteinMPNN samples — no HPC endpoint was attached.
- A pLDDT move from 81.66 to 81.79 is inside the noise. The loop is real; the science is a demo.
- This run had no ANTHROPIC_API_KEY. Every node took its deterministic path.

The point of the demo: a prompt in a browser produced a provenance graph, a scored ensemble and a training-ready Parquet set, with no human in the middle and nothing stubbed.

Three questions this deck is organised around

FUNCTIONALITY

The specific new tasks the software makes possible.

A chat turn plans, runs and records a redesign campaign

37 tasks, 12 designs, 114 scores in one session

A nine-stage cluster protocol, advanced one stage per turn

what it answers

PERFORMANCE

Runtime efficiency and effectiveness — including where scale and scientific yield are not linear in each other.

The chat answers in milliseconds while the work takes minutes

mean 2.04 tasks in flight, peak 6, over 146 s

One measured curve: what a file costs to cross the broker

what it answers · and what it does not

USABILITY

Not the interface — the use cases the software lends itself to.

As an application: a chat with a structure viewer

As a platform: per-user keys, 22 HTTP routes, a CLI

As a component: one contract, four interfaces, a readable lake

what it answers

The middle column is the weak one, and I would rather name it here than defend it later. One host, one process pool of four, one campaign, no GPU of our own. There is no scaling study in this deck, mean in-flight depth is not a speedup, and the act on performance therefore ends with a slide listing the measurements that do not exist — because this room can tell the difference between a measurement and an architecture diagram.

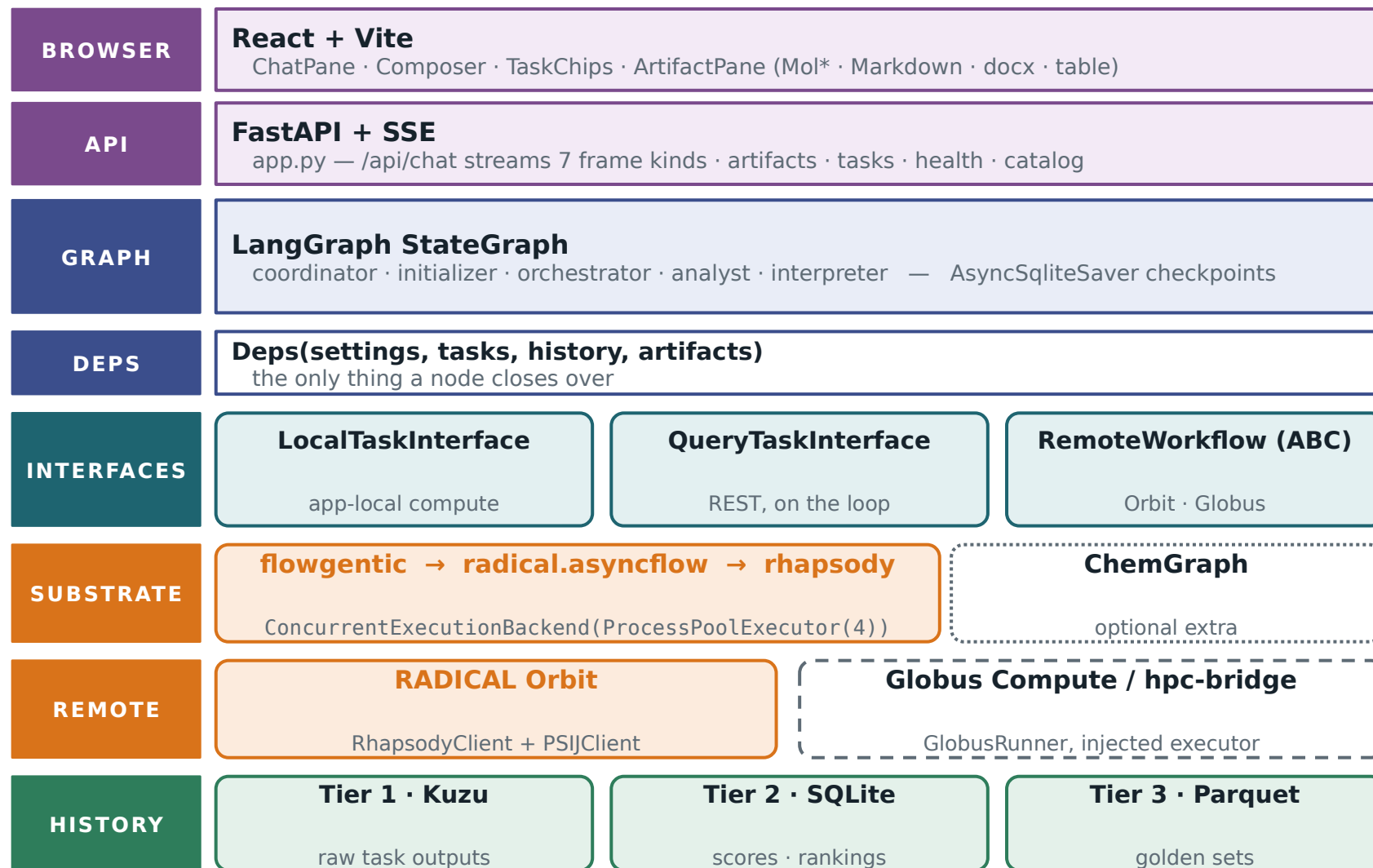
One constraint sets the whole design

A chat interface must answer in milliseconds. A protein fold takes tens of seconds. An HPC job takes hours, and may sit in a queue first.

consequence	what it means in the code	where
Tasks return futures, never results	Every interface's submit() places the work and returns a handle. Nothing in the graph awaits a task at submission time.	tasks/base.py:159
Task duration is not the interface's business	The same handle covers a 40 ms REST call and a queued batch job. Only the capability flags differ.	tasks/base.py:62
Status is a stream, not a return value	Nodes emit progress over LangGraph's custom channel; the manager fans task events onto the same SSE stream.	graph/deps.py · app.py:437
State holds references, not payloads	Coordinates go to a content-addressed blob and travel as a path, because every checkpoint is serialized.	orchestrator.py:292
Every layer has a floor	No key, no endpoint, no graph DB: each degrades to something that still answers, and says so.	runtime.py · analyst.py:166

Nothing below this slide is a free choice. If you drop the constraint — if the agent may block — most of the Task Interface layer stops paying for itself and you would call the tools directly.

Architecture, and where the RADICAL stack sits



The one rule

- A node reaches the outside only through Deps — never an interface or a store directly.
- That is what makes 417 of 441 tests run with no network, no pool and no endpoint.
- Convention, not an enforced check.

Where it leaves the process

- LocalTaskInterface is the only band that crosses into asyncflow.
- Everything above it is ordinary async Python, and stubbable.

- built, runs end to end
- built and tested, never run for real
- - - designed for, not implemented
- RADICAL component**

ACT 1 OF 3

FUNCTIONALITY

The specific new tasks a piece of software makes possible — what you can do now that you could not before.

One prompt runs a redesign campaign end to end

classify → initialize → propose → fold → score → rank → summarize, every step recorded: 37 tasks, 12 designs, 114 scores, 7 artifacts in the session on the next slide

A multi-day cluster protocol becomes a conversation

nine stages, one per turn, three of them stopping for a human — and the turn boundary is the checkpoint, because there is no interrupt() anywhere in this repo

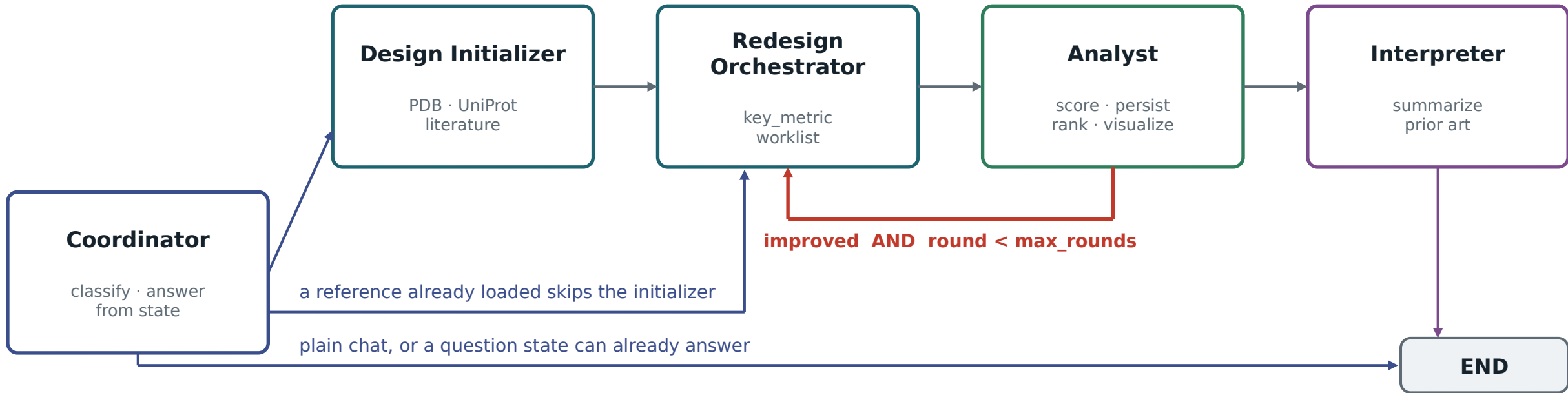
A campaign becomes a record the next campaign can query

three tiers behind one facade; `best_designs(exclude_campaign=...)` is how the interpreter finds prior art without rediscovering its own designs

The weakest claim in this act: the protocol's nine stages have never run on a cluster. 203 offline tests, zero real runs — so every resource figure in `protocol/specs.py` is an estimate.

Six nodes, and the nodes do their own routing

F1 is the campaign loop. The sixth node, protocol, answers in place and goes straight to END — it is slide 10.



```

destinations = {
  "coordinator": ("initializer", "orchestrator", "analyst",
                 "interpreter", "protocol", END),
  "initializer": ("orchestrator", END),
  "orchestrator": ("analyst", "interpreter", END),
  "analyst":      ("orchestrator", "interpreter", END),
  "interpreter":  (END,),
  "protocol":     (END,),
}
builder.add_edge(START, "coordinator") # the only static edge
  
```

graph/build.py:147–158 · VERBATIM, two comment lines elided

Why no conditional edges

- A node returns `Command(goto=..., update=...)`: the decision and the state write are one atomic return.
- `destinations=` is a declaration for validation and the drawn graph, not control flow.

State, its reducers, and what must never enter it

```
class DesignState(TypedDict, total=False):
    messages: Annotated[list, add_messages]

    reference_design: Annotated[ReferenceDesign, replace]
    key_metric: Annotated[KeyMetric, replace]
    worklist: Annotated[list[WorkItem], merge_worklist]
    lead_design: Annotated[Design, replace]
    ensemble: Annotated[list[Design], replace]
    molecular_visualization: Annotated[MolVisualization, replace]
    artifacts: Annotated[list[ArtifactRef], merge_artifacts]
    design_summary: Annotated[str, replace]

    # control
    intent · round · session_id · goal · status
    warnings: Annotated[list[str], merge_warnings]
    target_hints · requested_mutations · pending_results
```

graph/state.nv:211–240 · TRIMMED

Four reducers, each for a reason

- merge_artifacts — append, dedupe by id, last write wins: the analyst replaces a viz each round.
- merge_worklist — upsert by id, so status can be updated in place.
- merge_warnings — accumulate, dedupe, keep 20. Survives the turn; status does not.

Reducers are the only place concurrency shows up in state. Nodes run sequentially here, but artifacts and warnings arrive from different nodes in the same turn, and both must accumulate rather than clobber.

The finding: a checkpoint is serialized on every turn

First build put fold coordinates in pending_results. Six PDB files is ~320 KB of checkpoint per turn, and it grows with the ensemble. The orchestrator now writes coordinates to a content-addressed blob and passes a path:

```
summary["structure_path"] = deps.history.write_blob(
    structure, suffix=".pdb", prefix=summary["design_id"])
```

orchestrator.py:292–296 · TRIMMED

```
def test_structures_are_not_carried_in_state(...):
    blob = json.dumps(jsonable(out), default=str)
    assert "ATOM " not in blob
```

tests/test_graph.py · TRIMMED

Pinned by a test, because the regression is invisible: everything still works, just slower every turn.

Design History: three tiers, and what is actually in them

Tier 1 — raw task outputs

Kuzu, embedded graph DB

- 6 node tables · 8 rel tables
- Campaign 1 · Reference 1 · Design 12
- Task 37 · Output 37 · Structure 11
- properties columns hold JSON, so a new task needs no migration
- single writer behind an RLock

Tier 2 — scores, rankings, analysis

SQLite

- 114 score rows over 10 metrics
- 17 ranking rows (round 1: 6, round 2: 11 cumulative)
- 2 round analyses, with the failures list
- best_designs() can exclude the current campaign — that is how the
- interpreter finds prior art without finding itself

Tier 3 — golden sets

Parquet + manifest

- 11 rows, one metric_<name> column per metric seen
- rules: metric=plddt, direction=max, top_k=12,
- dedupe_sequences=true → 12 designs became 11 rows
- the manifest records the rules, so a set is reproducible
- staged for ML training, not for reading

```
def record_task_result(self, campaign_id, task_id, *, name, interface, state,
                      result=None, error="", designs=None, round_no=0,
                      reference_id=None, blob=None) -> dict:
    """Write one task's outcome across tiers 1 and 2.

    `blob` is an optional (data, suffix) pair for bulky raw output.
    Returns {"output_id", "blob_path"} for the caller to reference.
    """
```

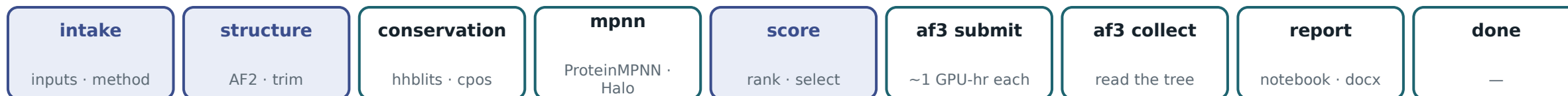
lake/store.py:89–108 · TRIMMED

Worth noticing: only tier 1 needs a running process to read. Tiers 2 and 3 are a SQLite file and a Parquet file, so the figures on this slide were produced by a script that opened them directly while the server held the Kuzu lock.

One facade, three stores

- DesignHistory is the only thing nodes see; tiers 1 and 2 are written together or not at all.
- Bulky output goes to a content-addressed blob (sha256[:16]) — 24 files, 5.1 MB on disk.

A multi-day cluster pipeline, one stage per turn



Indigo = the stage asks the user something and stops. Four questions, three stages, and no interrupt() anywhere in this repo: **the turn boundary is the checkpoint mechanism.**

```
protocol = state.get("protocol") or {}
if protocol.get("awaiting") and not _wants_out(text):
    return Command(
        goto="protocol",
        update={"intent": "protocol",
                "status": "Continuing the protocol..."},
    )
```

coordinator.py:269–274 · VERBATIM, one line wrapped

It has to come before classification: "liu", "310,364" and "go" all classify as chat, which answers from session state and leaves the campaign waiting forever. ESCAPE_WORDS is the explicit way out.

Two job modes, and the choice that matters

- Transfer specs declare inputs/outputs, so artifacts.wrap rewrites them into a mktmp -d script on the local executor: no queue slot.
- Compute specs declare neither and set directory under \$PROJ, so their files persist for the next stage and their stdout is the chip's log tail.

Two tables, so each entry is testable alone

- STAGES: stage → what it runs. ANSWERS: an awaiting key → how the reply is read. A stage sets awaiting and returns; the next message answers it.
- It is the only node that replies every turn and never routes onward — and the only one whose state has to survive the checkpoint to be useful.

specs.py is the source of truth

- The skill's #SBATCH headers are dead here: PSI/J generates its own submit script. Every walltime, core count and memory figure lives in protocol/specs.py.
- ResourceSpecV1.memory is in bytes: {"memory": 32} renders --mem=0K, which Slurm reads as the whole node. Everything goes through gib().

Progress is read from the filesystem, never from a handle. Handles are in-memory, so a restart cannot re-attach; af3_collect lists the output tree instead, which makes it idempotent and identical whether the submit was minutes or days ago. 203 of the 417 offline tests are this node. No stage has run on a cluster yet.

ACT 2 OF 3

PERFORMANCE

Runtime efficiency and effectiveness — and the places where scale and scientific yield are not linear in each other.

The chat answers in milliseconds while the work takes minutes

mean 2.04 tasks in flight over 146.1 s, peak 6, longest single task 42.4 s — nothing in the graph awaits at submission time

The seam is what buys that, and it costs one contract

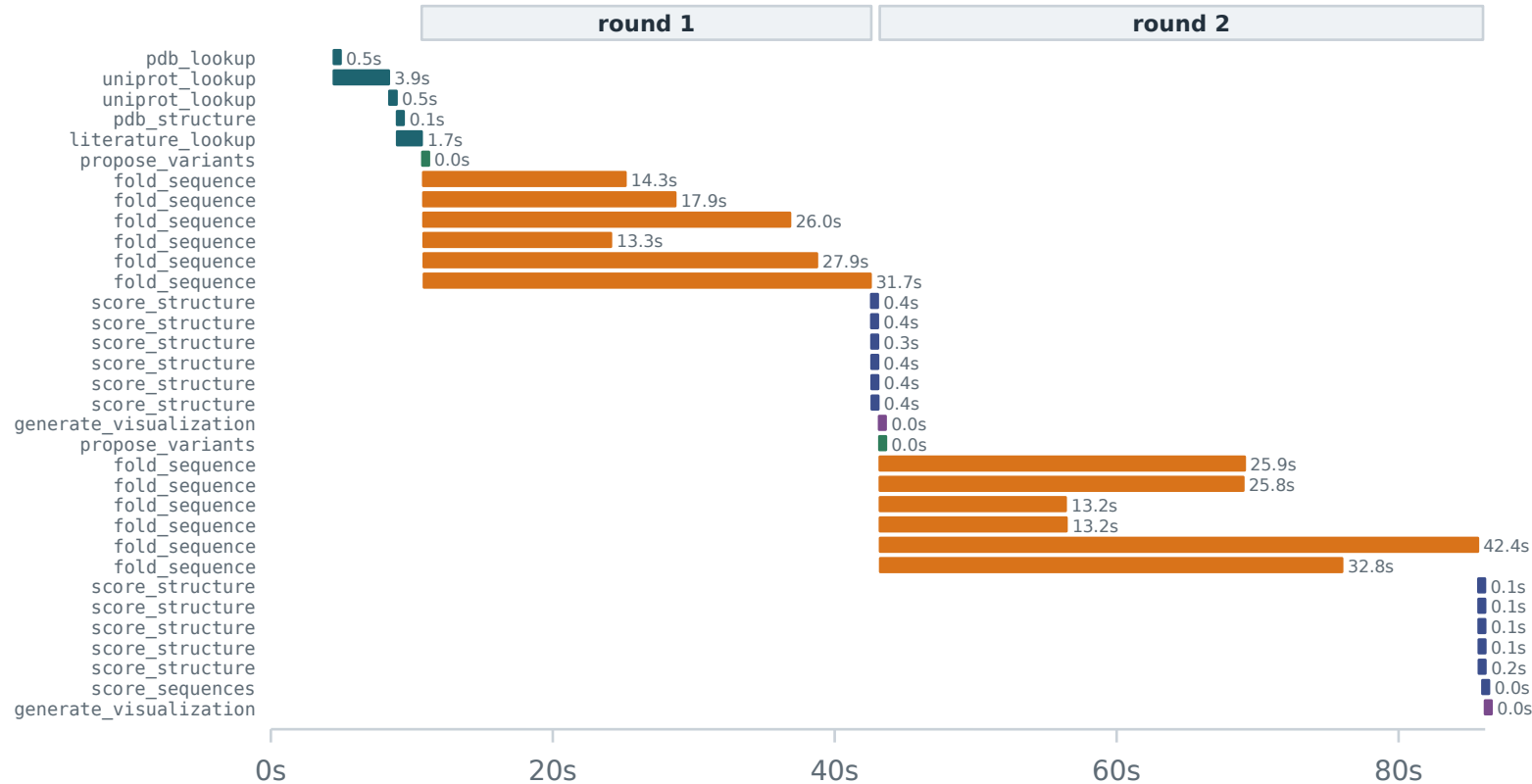
five verbs, four implementations; a failed submission becomes a settled FAILED handle rather than an exception, so a caller has one shape to handle and gather() never raises

One number here is a curve, and it was measured for this talk

what a file costs to cross the broker: $\times 1.33$ of stdout per byte for incompressible data, $\times 0.33$ for structure text

The weakest claim in this act: there is no scaling study. One host, one pool of four, one campaign — mean in-flight depth is not a speedup, and this act ends with the slide that says exactly which measurements are missing.

One campaign, measured: 37 tasks in 146 seconds



37 tasks · 297.9 s of handle lifetime in 146.1 s of wall clock · mean 2.04 in flight, peak 6

Derived from these rows at build time, not transcribed. It is also not a speedup: a handle's lifetime includes time queued inside a pool of four, and this deck has no serial baseline to divide by. The critical path is one 42.4 s fold.

Read the fold rows

- Six submitted in the same instant, finishing 13.3 s to 31.7 s later — out of order, as they land.
- Round 2 ran 13.2 s to 42.4 s. flowgentic's 30 s default timeout would have killed half of them.

Read the lookup rows

- pdb_lookup and uniprot_lookup go out together; a cross-reference follow-up resolves second.
- Then structure download and literature, also concurrent.

And one real failure

- ESM Atlas dropped one of round 2's six folds: "s-tutp87hw-r2-5: ESM Atlas returned no structure".
- That design survived as a sequence-only entry with 4 metrics; the round scored the other 5 and the user got a warning.

★ The Task Interface contract

```
class TaskInterface(ABC):
    name: str = "base"
    capabilities: Capabilities = Capabilities()

    @abstractmethod
    async def submit(self, spec: TaskSpec) -> TaskHandle:
        """Place the task and return immediately."""

    async def status(self, handle) -> TaskState
    async def logs(self, handle, offset=0) -> LogChunk
    async def result(self, handle) -> Any
    async def cancel(self, handle) -> bool
    async def close(self) -> None
```

tasks/base.py:148–186 · TRIMMED

```
@dataclass(frozen=True)
class Capabilities:
    supports_cancel: bool = False
    supports_log_stream: bool = False
    supports_push_events: bool = False
    supports_staging: bool = False
```

tasks/base.py:62–67 · VERBATIM

interface	what it is	cancel	logs	push	stage
local	app-local compute, rhapsody pool	✓	—	—	—
query	REST retrieval, on the loop	✓	—	—	—
orbit	Rhapsody tasks + PSI/J jobs	✓	✓	✓	✓
globus	Globus Compute (hpc-bridge)	part	—	—	—

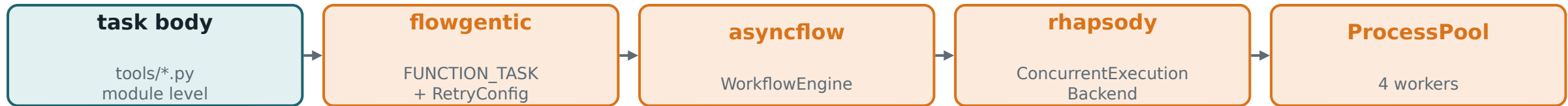
The flags are the honest part. An interface that cannot tail logs says so, and the manager simply does not start a drain for it — no caller has to know which backend it got.

```
try:
    handle = await interface.submit(spec)
except Exception as exc:
    # Surface the failure as a settled handle so callers
    # need no separate error path.
    future = loop.create_future()
    future.set_exception(exc)
    handle = TaskHandle(..., state=TaskState.FAILED,
                        error=str(exc))
```

tasks/manager.py:134–150 · TRIMMED

The invariant: a caller holds a handle whose future resolves. Not sometimes a handle and sometimes an exception. A rejected submission is a handle that has already failed — which is why gather() never raises and the round survives a dead endpoint.

★ Through flowgentic to a rhapsody process pool



```

retry = RetryConfig(
    # Folding or an HPC job can take far longer
    # than flowgentic's 30s default.
    timeout_sec=None,
    max_attempts=1,
    retryable_exceptions=(ConnectionError, OSError),
)
def wrap(body):
    return integration.execution_wrappers.asyncflow(
        body, flow_type=AsyncFlowType.FUNCTION_TASK,
        backend="compute", retry=retry)
  
```

runtime.py:295–308 · TRIMMED

```

async def submit(self, spec: TaskSpec) -> TaskHandle:
    fn = self._resolve(spec.name)
    # create_task so submission returns immediately: the
    # flowgentic wrapper for FUNCTION_TASK is a coroutine
    # function, not a future factory.
    future = asyncio.ensure_future(
        _call(fn, spec, llm_credentials(spec.name)))
    return self._handle(spec, future)
  
```

tasks/local.py:80–85 · VERBATIM

Three constraints the pool imposes

- Task bodies live at module level in tools/ — no closures over clients, or they will not pickle.
- Clients are built inside the body; a parent's cannot cross into a worker.
- The entry point needs a `__main__` guard, or uvicorn re-imports under fork.

Finding 1 — flowgentic hard-imports aiohttp

```

# Try to include aiohttp timeouts if present
try:
    import aiohttp # type: ignore
    ex.extend([aiohttp.ClientConnectionError,
               aiohttp.ServerTimeoutError])
except Exception:
    raise
  
```

refcodes/flowgentic/.../fault_tolerance.py:73–84 · VERBATIM

The comment says "if present"; the code says "or die". Same shape for httpx at :60–72. Reached whenever `retryable_exceptions` is left at its default, which is `()`.

★ RADICAL Orbit: two clients, two concurrency bridges

EndpointRuntime(broker_url, cert, name, token)

RhapsodyClient

function + executable tasks
register_session · submit_tasks
wait_tasks · get_task · cancel_task

pushes task_status events

PSIJClient

batch jobs
submit_job · cancel_job
get_job_status(id, stdout_offset,
stderr_offset)

the only log tail that exists

Two threading problems, two one-liners

- Every Orbit client method is synchronous and blocking → each of 23 call sites goes through `asyncio.to_thread`.
- Push callbacks arrive on Orbit's listener thread → `_dispatch` hops to our loop with `call_soon_threadsafe`.

```
def _dispatch(self, fn, *args) -> None:
    """Hop from Orbit's listener thread onto our event loop."""
    if self._loop is None or self._loop.is_closed():
        return
    self._loop.call_soon_threadsafe(fn, *args)
```

tasks/hpc/orbit.py:328–332 · VERBATIM

Findings 4 and 5 — what a terminal event does not carry

```
# The terminal event carries state and exit_code but not
# stdout, so the full record is fetched before the future
# is resolved.
if not data.get("stdout") and self._rhapsody is not None:
    info = await asyncio.to_thread(
        self._rhapsody.get_task, handle.id)
    # The event wins on state; the fetch fills in the output.
    merged = {**info, **{k: v for k, v in data.items()
                        if v not in (None, "")}}
```

tasks/hpc/orbit.py:353–366 · TRIMMED

```
if state is TaskState.FAILED and not error:
    error = (f"exit code {exit_code}" if exit_code not in (None, 0)
            else (data.get("stderr")
                  or handle.log_tail[-500:] or "job failed"))
```

tasks/hpc/orbit.py:428–433 · TRIMMED

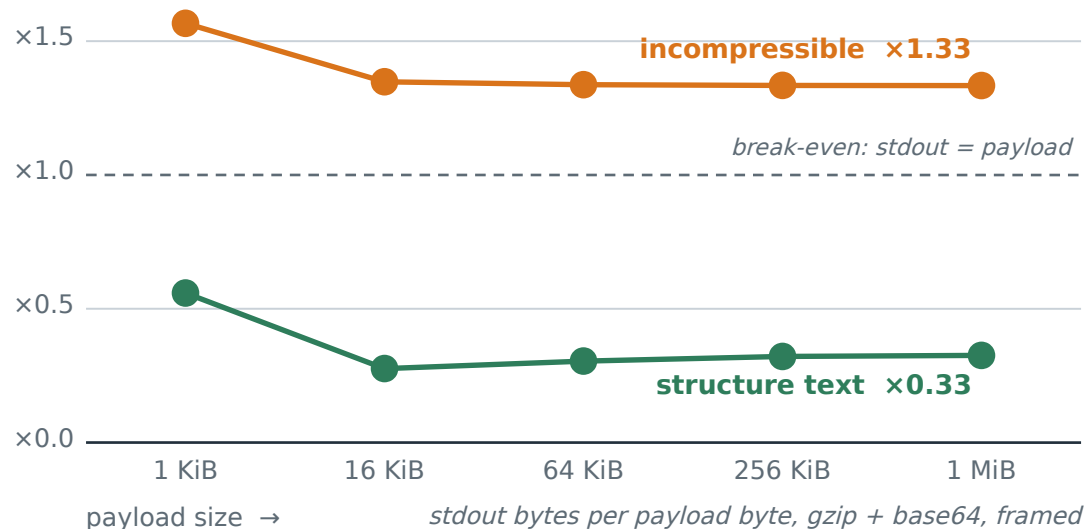
A completed task's result came back empty until the re-fetch. A job that exited 3 reported no reason at all — only the code — so FAILED always synthesises an explanation.

How far this is proven

- 12 tests against a real localhost broker + endpoint: push states, incremental log tailing by offset, a failing job, cancelling a running one, that stdout comes back whole and unduplicated, that declared inputs and outputs actually travel, and one real ProteinMPNN run on CPU.
- And, since 2026-10-09, 8 more against a real one: a broker on a public VM, an endpoint registered from a cluster login node, Slurm ids back in `handle.meta`. Slide 19.

What we have measured, and what we have not

F5 · what one file costs to cross the broker



Both ceilings are in the wrong units

- Outbound: a 2 MiB PDB is refused as too_large — the cap is checked on the raw size — though it compresses to 0.67 MB. A 1 MiB random file is accepted, and costs 1.40 MB.
- Inbound has no declared cap at all: the chunks must fit ARG_MAX, so exec fails with E2BIG at ~1.5 MiB of incompressible payload. Our backlog, not an ask.

Measured, and in this deck

- mean 2.04 tasks in flight over 146.1 s, peak 6 — derived from run.json at build time
- the critical path is one 42.4 s fold; six folds share four pool slots, so two of each round wait
- ~320 KB of checkpoint per turn before coordinates moved to blobs — growing, and breaking nothing
- the staging channel, left: measured against the real wrap() and collect()
- 417 offline tests in ~155 s with no network, no pool and no endpoint

Not measured — so not claimed

- no scaling curve: pool width has never been varied, and 4 is a default, not a finding
- no GPU occupancy. ORBIT_JOB_GPUS=1 is a request; nothing here has read a GPU's utilisation
- no multi-node run, and never two endpoints at once
- no serial baseline, so nothing above divides into a speedup
- every walltime and memory figure in protocol/specs.py is still an estimate

Where scale and science stop being linear: stage_score prints “AlphaFold3 is about an hour of GPU each, so that is N GPU-hours” and stops for an answer (protocol.py:616). Cost is linear in the designs you fold; yield is not — they are diversity-selected, so the tenth is less like the other nine than the second was. That trade is the one checkpoint in the protocol a human cannot skip.

Degradation is a feature, and it is tested

when	what happens	evidence
No ANTHROPIC_API_KEY	every node has a deterministic rule-based path	the measured campaign ran this way
No Orbit endpoint	interface_for falls hpc → local and logs it	tests/test_tasks.py
ChemGraph not installed	the task reports unavailable; nothing else changes	chemgraph_available()
ESM Atlas drops a request	design survives sequence-only; round scores the rest	really happened: s-tutp87hw-r2-5
Tier 1/2 write fails	warn, rank in memory, tell the user in the reply	test_lake_write_failure_does_not_lose_the_round
No SQLite checkpointer	InMemorySaver, with a note on /api/health	runtime.py:466

```
# A storage failure must not lose the user's round: the designs are
# already in state, so we log, warn, and carry on.
try:
    ranked = deps.history.rank_round(campaign_id, round_no, ...)
except Exception as exc:
    storage_error = storage_error or str(exc)
    log.warning("persisting the ranking failed: %s", exc)
    ranked = _rank_in_memory(combined, metric_name, direction)
...
if storage_error:
    update["warnings"] = [f"Design History write failed: {storage_error}"]
```

graph/nodes/analyst.py:166–233 · TRIMMED

How the user finds out

- The interpreter appends a "Caveats from this run" section to its reply, from the warnings channel.

Where this came from

- A readonly-SQLite error during a live run. My own fault — but it exposed unguarded writes.

USABILITY

Not the interface — the use cases: what this lends itself to as an application, as a platform, and as a component in someone else's workflow.

As an application: a chat with a structure viewer

seven SSE frame kinds in one ordered stream, task chips that name their interface, and artifacts — Markdown, .docx, a sortable ensemble table, two Mol* specs — you can hand to a collaborator

As a platform: other people's keys, endpoints and allocations

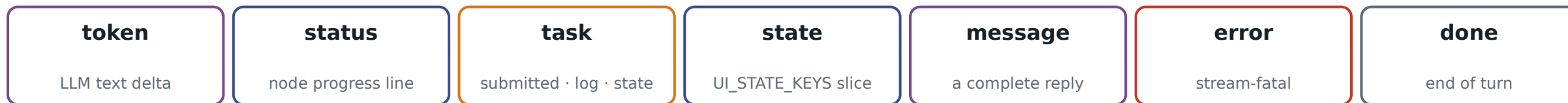
logins, per-user credentials as Fernet ciphertext, one OrbitInterface per user under its own client name; 22 HTTP routes and a CLI, so no part of it needs the browser

As a component: one contract, and a record that outlives the process

the Task Interface is the extension point — four implementations over ~102 shared lines — and tiers 2 and 3 of the lake are a SQLite file and a Parquet file, readable with nothing running

The weakest claim in this act: one deployment, one lab, and no second consumer. Nothing has yet driven this as a component except its own tests — the component claim is the shape of the code, not evidence.

The web app: seven frame kinds and one viewer



POST /api/chat returns text/event-stream. One asyncio.Queue merges the graph's own astream frames with TaskManager events, so task progress and node status share one ordered stream.

```
let buffer = "";
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  let split;
  while ((split = buffer.indexOf("\n\n")) !== -1) {
    const raw = buffer.slice(0, split);
    buffer = buffer.slice(split + 2);
    ... yield JSON.parse(line.slice(6)) as Frame;
  }
}
```

A chunk boundary can land mid-frame, so partial frames are held in a buffer until a blank line. A malformed frame is skipped, never fatal.

Confirmed rendering in a browser on 2026-10-01. It was built without one — the dev server, the proxy, a real artifact through it and the CDN assets were all that could be checked at the time.

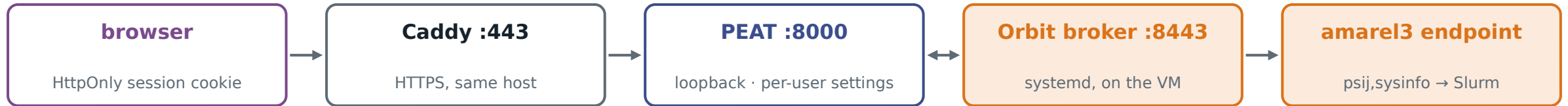
The Mol* viewer

- rcsb-molstar 2.14.7 UMD from jsDelivr, loader cached on window.__molstarLoader so one fetch serves every mount.
- One viewer per mount; the spec is re-applied on change; ResizeObserver → handleResize.

Two things that bit

- createComponent takes no colour, so colours go through pluginCall → updateRepresentationsTheme.
- Coordinates are inlined as a json artifact by the analyst, so the viewer needs no second authenticated fetch.

Deployed, and every user brings their own keys



one VM: Caddy, the agent and the broker

the endpoint is a cluster login node, across the internet

Logins are off by default, and off they change nothing

- No `runtime.auth` → `_user()` returns `None` and every ownership check passes, which is why the other 399 tests needed no edit when logins arrived.
- On: every route but login, logout and a bare `/api/health` needs the cookie; a session must be the caller's (404 otherwise); settings need the admin role, not a shared token.

```
def settings(self) -> Settings:
    turn = current_turn.get()
    return turn.settings if turn is not None else self.base_settings
```

graph/deps.py:49–51 · VERBATIM

A turn's user travels in a `ContextVar`, which `asyncio` copies into every task the turn creates. So every node gets that user's key and endpoint without knowing users exist, and each user's endpoint gets its own `OrbitInterface` under its own client name.

A key that must not be written down anywhere

- Not in `configurable`: `LangGraph` copies every string in it into checkpoint metadata, so the key would be saved every turn.
- Not in `spec.params`: those are written to the lake and returned by the task routes. A pool task is handed the key as a call argument instead.
- `test_no_user_secret_is_persisted_by_a_turn` scans the checkpoint, the task snapshots and the data dir for it.

What this does not give you — backlog A21

- The broker is shared, and it is not a tenant boundary: one ingress token, and anyone holding it can submit to every endpoint on it. Per-user HPC isolation needs a broker per user.
- Sessions from before logins have no owner, so they are unreachable under logins.
- The lockout is in memory, and trusts `X-Forwarded-For` only from a loopback peer — right behind this proxy, wrong behind any other.

Running it, driving it, and what each tier proves

```
$ ./scripts/setup.sh                # builds .venv, then verifies it
$ python -m designagent --reload      # :8000
$ python -m designagent --check-config --probe # config, each credential tried
$ python -m designagent --add-user NAME --admin # and --gen-secrets-key
$ cd frontend && npm install && npm run dev # :5173, proxies /api
$ pytest -q                          # 417 tests, no network
$ pytest -q -m live                   # 12 tests, starts a real broker
$ pytest -q -m remote                 # 8 tests, a real endpoint
```

VERBATIM

Driving it without the browser

- 22 HTTP routes: /api/chat is the SSE stream, the rest REST — sessions, tasks, artifacts, settings, catalog. Plus the CLI above.
- The record outlives the process: tiers 2 and 3 are a SQLite file and a Parquet file, read by run_model.py under the Kuzu lock.
- uv sync cannot work here — four editable installs from refcodes/, and flowgentic pins two of its deps to git URLs.

file	n	what it actually proves
tests/test_protocol_*.py	203	the validators between a chat message and a shell script, every job spec run under bash, the stage machine turn by turn
tests/test_graph.py	55	18 classifier cases, a full loop end to end, artifacts, lake-write failure, no structures in state
test_tasks · _jobspec · _proteinmpnn	70	the contract; the job spec, in-band staging and its ceiling, the FASTA adapter
test_auth.py · test_settings.py	46	off, logins change nothing; a session belongs to its owner; everything writable is reported
test_lake · _api · _artifacts	43	all three lake tiers against a tmp_path; SSE frames; cancel → 409 with a reason
-m live · -m remote · -m llm	12·8·4	a broker and endpoint as subprocesses · a real scheduler · a real API key

The split that matters: 417 tests need no network, no pool and no endpoint, because nodes only reach through Deps. The 24 that do are marked live, remote and llm — each deselected separately, so the cheap tier can never pull in one that needs an allocation or spends money.

What is real, and what is not — in all three

FUNCTIONALITY	PERFORMANCE	USABILITY
Runs end to end against something real The campaign loop, all six nodes two redesign rounds, routed by the improvement test, on a real target All three lake tiers 114 scores, 37 task nodes, an 11-row Parquet set Artifacts and streaming Markdown, .docx, a sortable table, two Mol* specs — pane confirmed in a browser	Runs end to end against something real Work really leaves the process 12 real ESMFold predictions on the rhapsody pool, reaped out of order Orbit against a real cluster 2026-10-09: Slurm jobs through a public broker, logs, a cancel, native_id back ProteinMPNN, the real model v_48_020 on CPU through a real broker; designs carry its score and hash	Runs end to end against something real As an application, deployed a VM behind Caddy, sign-in, the viewer driven in a real browser As a platform per-user logins and credentials encrypted at rest; each user's own endpoint As a component, in shape 22 HTTP routes and a CLI; lake tiers 2 and 3 read with nothing running
Built, and has never run for real No protocol stage has run on a cluster nine stages, 203 tests, zero real runs — specs.py is a guess A restart during a protocol run handles are in-memory: recovery is a re-fetch, nothing re-attaches ChemGraph wired behind the interface, exercised by tests, never by a campaign	Built, and has never run for real No scaling study of any kind one host, a pool of four never varied, no multi-node, no occupancy No GPU of our own every AF3 walltime and memory figure is an estimate until a queue says otherwise wrap_nodes ships off, against the approved design. That is the ask on the next slide	Built, and has never run for real A broker per user one shared ingress token, so HPC is not yet a tenant boundary Driven by anything but itself no second consumer — the component claim is the code's shape, not evidence The Globus adapter structurally complete, tested with an injected executor, never run live

Three red columns as long as the three green ones, on purpose. The single most load-bearing caveat is still the first one: no stage of the protocol has run on a cluster. The path under it works — real jobs, real logs, a real cancel — and the performance column is the one I would attack if I were you.

Eight reproducibles, by what each one costs

A WRONG ANSWER, OR REAL TIME LOST

- 3 Orbit's terminal task event omits stdout** functionality
A completed task resolves with an empty result until `get_task` is called again. Worked around in `_finish_task_enriched`.
- 4 A FAILED Orbit job carries no reason** functionality
Only a non-zero exit code reaches the client, so FAILED synthesises an explanation from code, then `stderr`, then the log tail.
- 5 Neither outputs nor `stdin_text` is forwarded** performance
So `stdout` is a job's only channel for a file. Hence in-band staging — and the curve two slides back is what that costs.
- 2 RetryConfig defaults cancel long tasks silently** performance
30 s per attempt, 3 attempts. This repo's own folds take 13–42 s, so half of round 2 would be cancelled and re-run.

PAID BY WHOEVER ADOPTS THE STACK NEXT

- 6 `asyncflow` swallows SIGTERM** usability
It installs its own handler, reports a completed shutdown, and leaves the process running — so every stop has to escalate.
- 1 `flowgentic` hard-imports `aiohttp` — and `httpx`** usability
`fault_tolerance.py:71–72` and `:83–84` are except `Exception: raise` where the comment says "if present".
- 7 The broker has no HTTP topology route** usability
`/topology` → 307 → 404, read as a plugin name. Readiness has to come from `rt.topology()`, which propagates asynchronously.
- 8 `--no-auth` still requires cert and key** usability
It disables ingress auth only; the broker always serves TLS and refuses to start without a pair. One line in the docs.

The question: is `flowgentic`'s `EXECUTION_BLOCK` meant to preserve the caller's context?

A node wrapped as one runs on `asyncflow`'s loop, outside `LangGraph`'s runnable context. `get_stream_writer()` raises "Called `get_config` outside of a runnable context" and every custom event is dropped silently — the graph completes, the chat just goes quiet. If yes, it is a bug and `wrap_nodes=True` becomes our default. If no, `flowgentic`'s node wrapping and `LangGraph`'s streaming are mutually exclusive, and that belongs in the README.

The streaming experiment, in full

What was tried

- The approved design: wrap every node as AsyncFlowType.EXECUTION_BLOCK with RetryConfig(timeout_sec=None, max_attempts=1), same as tasks.
- Symptom: the graph completed correctly and the chat showed no status lines at all. No error surfaced anywhere.
- Isolation: one plain node and one wrapped node, each calling get_stream_writer() and emitting one event.
- Plain node: writer obtained, event received by astream(stream_mode=["custom"]).
- Wrapped node: RuntimeError "Called get_config outside of a runnable context"; zero events.

Why

- flowgentic's EXECUTION_BLOCK submits the body to the asyncflow WorkflowEngine, which runs it on its own loop.
- LangGraph's get_stream_writer() reads the runnable context, which is a contextvar set by the LangGraph executor around the node call.
- Crossing to another loop loses the contextvar. The writer cannot be found, and LangGraph's own helper degrades to a no-op rather than raising — which is why it is silent.
- This is not a flowgentic bug unless EXECUTION_BLOCK intends to propagate context. Hence the question, not the accusation.

```
def wrap_node(integration, fn):
    """Wrap a node body as a flowgentic EXECUTION_BLOCK, if enabled."""
    if integration is None:
        return fn
    return integration.execution_wrappers.asyncflow(
        fn, flow_type=AsyncFlowType.EXECUTION_BLOCK, retry=node_retry_config())

# build_graph(): integration is set to None unless settings.wrap_nodes
```

graph/build.py:41–133 · TRIMMED

Verified after the change: 11 status lines stream for one redesign turn.

What ships

- wrap_nodes=False, the code path kept and documented.
- Tasks are still wrapped — FUNCTION_TASK runs task bodies, which never touch the stream writer.
- So the pool is genuinely used; only node bodies stayed in-process.

Standing up a local Orbit stack

```
# broker: --no-auth disables INGRESS AUTH ONLY. TLS is always served,
# and it refuses to start without a cert and key.
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem \
  -days 3 -subj /CN=127.0.0.1 \
  -addext subjectAltName=IP:127.0.0.1,DNS:localhost      # SAN must cover 127.0.0.1
chmod 600 key.pem                                     # refuses a looser mode

radical-orbit-broker.py --no-auth --host 127.0.0.1 --port N \
  --cert cert.pem --key key.pem

RADICAL_ORBIT_RHAPSODY_BACKEND=concurrent \
radical-orbit-endpoint.py --name local -p rhapsody,psij \
  --url https://127.0.0.1:N --cert cert.pem
```

tasks/hpc/local_orbit.py · EDITED for the slide

Readiness

- There is no HTTP topology route — /topology reads as a plugin name and 404s.
- The stack greps the endpoint's own log for registered as '<name>'.

Then the client decides

- OrbitInterface.connect() polls rt.topology() until the endpoint appears, bounded by connect_timeout — topology propagates asynchronously.

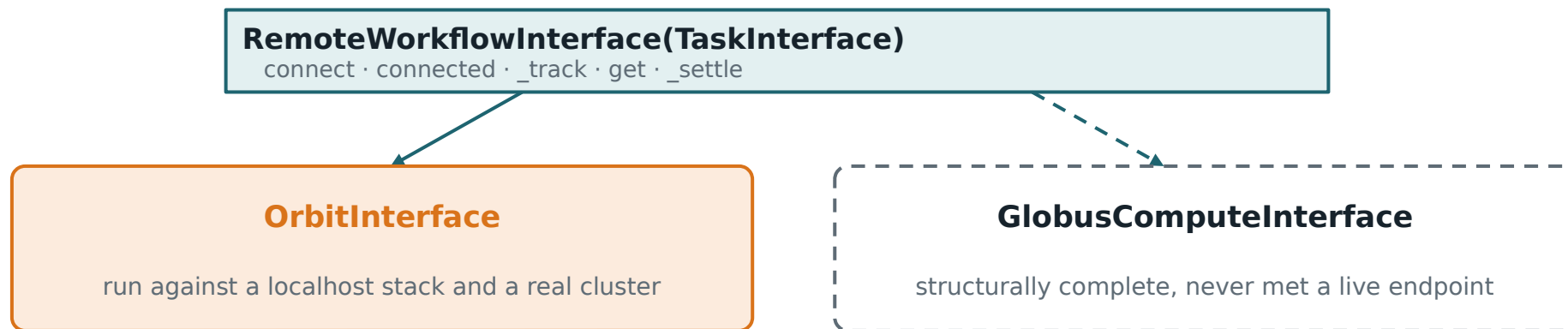
Why subprocesses, not EmbeddedBroker

- The embedded broker expects operator-placed credentials in ~/.radical/orbit, which a test must not create or touch.
- Subprocesses with a throwaway cert keep the whole thing inside tmp_path and leave no state behind.
- Cost: the stack has to detect readiness from logs, which is the fragile part of the fixture.

What the fixture then proves

- An executable task runs and its stdout comes back (after the get_task re-fetch).
- A bash loop's output tails incrementally — log_offset advances, so it is not refetched whole.
- A job exiting 3 lands FAILED with a non-empty error.
- A sleeping job cancels while running, and reports CANCELED.

Designed for two, implemented for one



What the abstraction cost

- Almost nothing: connect/_track/_settle plus drain_logs is the whole shared surface, and it is 102 lines.
- The interesting work is backend-specific by nature — Orbit's push callbacks have no Globus analogue.
- So the ABC's job is to make the differences declarable rather than to hide them.

The claim I will defend: the capability flags are more valuable than the shared base class. They are what let the manager and the UI behave correctly against a backend they were not written for.

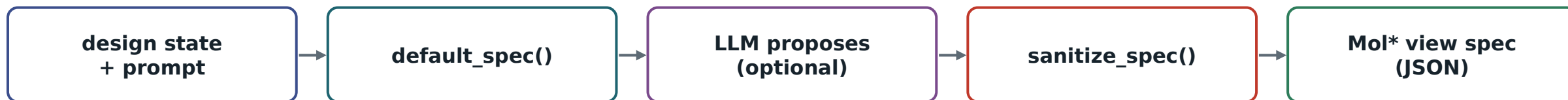
```
capabilities = Capabilities(
    supports_cancel=False,      # only before start; see cancel()
    supports_log_stream=False,  # final stdout only
    supports_push_events=False, # the future resolves; no
                                # intermediate states
    supports_staging=False,     # shared FS or Globus Transfer
)
```

tasks/hpc/globus.py:31–36 · VERBATIM

Testable with no endpoint

- GlobusRunner's executor_factory is injectable, so the suite passes a plain executor taking a callable.
- argv is a list run with no shell on that path; the real ShellFunction path uses shlex.join because its API takes a string. A test proves a metacharacter stays data.

Local Task Agents: a spec, not generated code



Why a spec and not generated JavaScript

- The brief said the agent "codes a browser-based visualization". Shipping LLM-written JS into the viewer is an injection surface for a prompt.
- So the agent emits a constrained JSON spec — structures, highlights, representation, colours — and the React component is the only thing that touches Mol*.
- `sanitize_spec` repairs or drops every field. A bad generation degrades to a plain cartoon, never a broken pane.
- Same expressive range for what a reviewer actually asks for; no execution.

```
def sanitize_spec(spec: Any, *, fallback: dict) -> dict:
    """Validate an LLM-proposed spec, repairing or dropping
    bad fields.
```

```
    Anything unexpected falls back rather than reaching the
    viewer, so a bad generation degrades to a plain cartoon
    instead of a broken pane.
```

```
    """
    if not isinstance(spec, dict):
        return fallback
```

tools/molviz_agent.py:143–150 · TRIMMED

agent	interface	what it does	status
Molecular Visualization Generator	local	Mol* view spec keyed from state + prompt	runs; drew both leads in the measured campaign
Cheminformatics Agent (ChemGraph)	local	ChemGraph single_agent workflow in a pool worker	wired, optional extra, exercised by tests only
ESMFold	local → hpc	ESM Atlas API under 400 aa, else a job	12 real predictions on disk
ProteinMPNN	hpc	PSI/J job spec + FASTA parser	no endpoint → heuristic proposer instead